30. BACKWARD CHAINING

AIM

To develop a Prolog program that implements backward chaining by starting with a goal and working backward through rules to verify whether the goal can be proved from known facts.

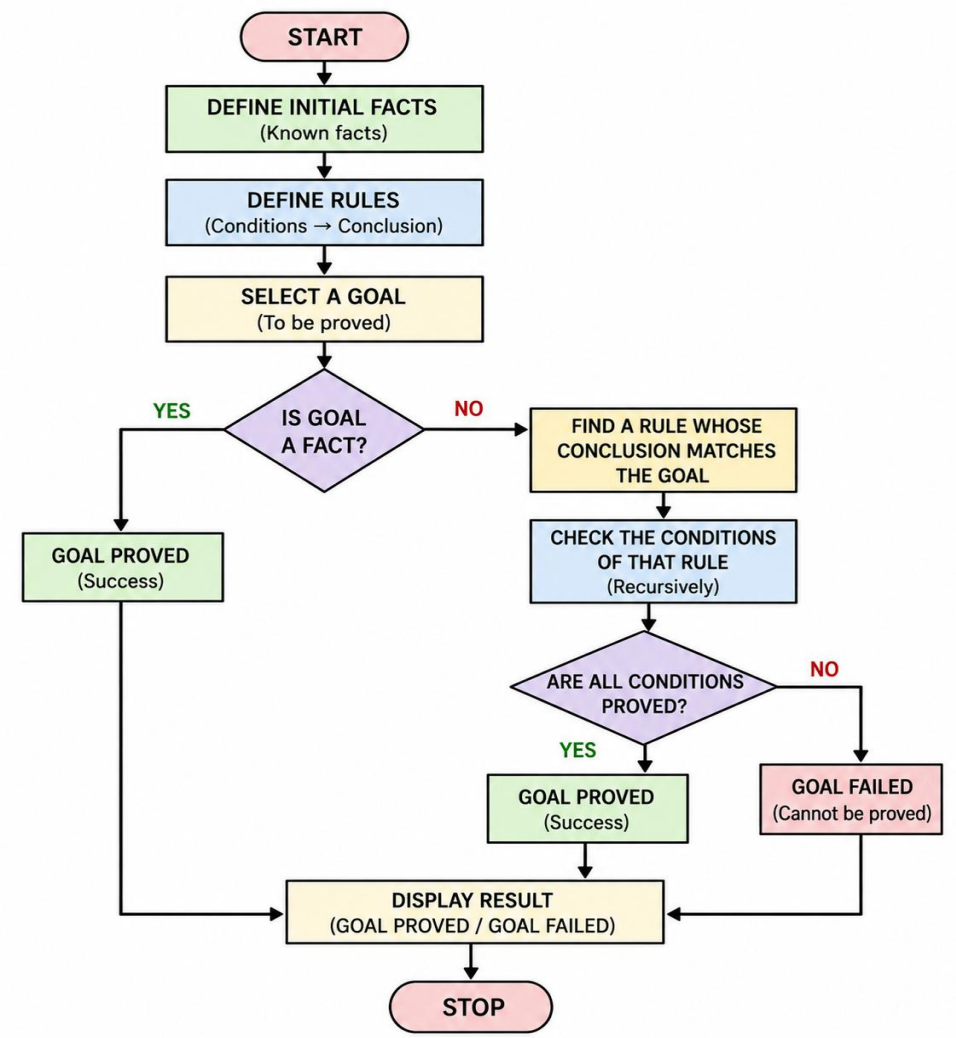
OBJECTIVE

- To understand the concept of backward chaining in Artificial Intelligence.
- To represent knowledge using facts and rules in Prolog.
- To start with a goal and search for supporting facts.
- To execute Prolog queries and verify the conclusion.

ALGORITHM

1. Start the program.
2. Define the initial facts.
3. Define the rules required to derive conclusions.
4. Select a goal to be proved.
5. Check whether the goal is directly available as a fact.
6. If not, find a rule whose conclusion matches the goal.
7. Check the conditions of that rule recursively.
8. If all conditions are satisfied, prove the goal.
9. Display the result.
10. Stop.

FLOWCHART

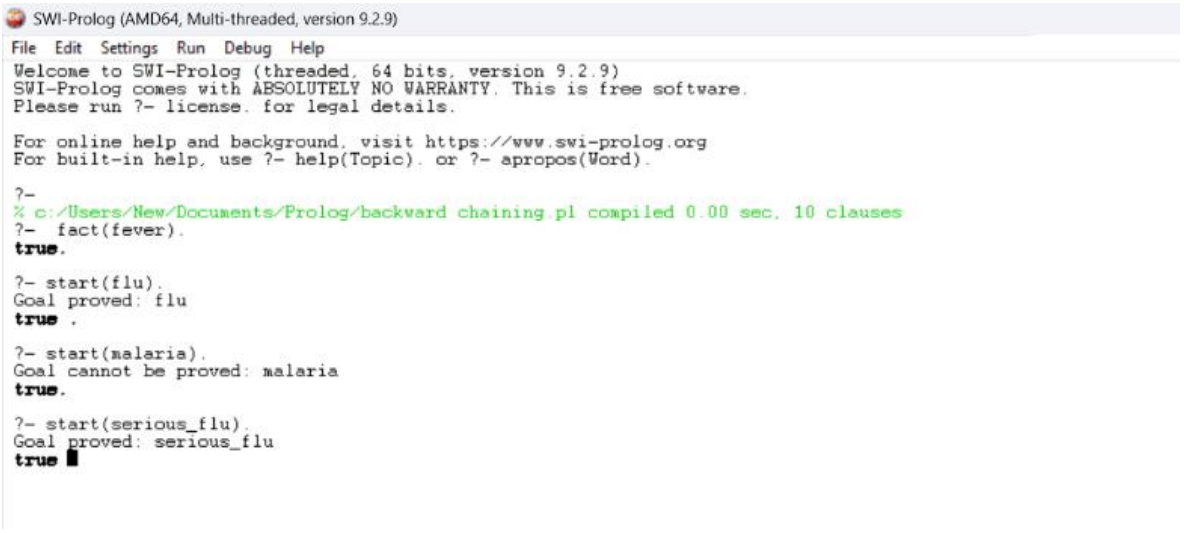


CODE

```
% BACKWARD CHAINING PROGRAM
% INITIAL FACTS
fact(fever).
fact(cough).
fact(body_pain).
% RULES
rule(flu) :-
    fact(fever),
    fact(cough),
```

```
    fact(body_pain).
rule(infection) :-
    fact(cough),
    fact(body_pain).
rule(serious_flu) :-
    rule(flu),
    rule(infection).
% BACKWARD CHAINING
backward_chain(Goal) :-
    rule(Goal).
backward_chain(Goal) :-
    fact(Goal).
% START PROGRAM
start(Goal) :-
    backward_chain(Goal),
    write('Goal proved: '),
    write(Goal),
    nl.
start(Goal) :-
    \+ backward_chain(Goal),
    write('Goal cannot be proved: '),
    write(Goal),
    nl.
```

OUTPUT



RESULT

Thus, the backward chaining program was successfully implemented in Prolog. The program starts with a goal and works backward through the available rules and facts to determine whether the goal can be proved.